

BCSentinel Go-Live Abnahmecheckliste

Finale manuelle Freigabecheckliste fuer Pilot, zahlenden Kunden und AppSource

Dokument	docs/MANUAL_GO_LIVE_CHECKLIST.md
PDF	docs/BCSentinel_GoLive_Checklist.pdf
Datum	2026-06-10
Status	Manual Execution Required

Inhaltsverzeichnis

1. Management Summary
2. Aktueller Projektstatus
3. Infrastruktur und Deployment
4. Billing und Stripe
5. Business Central Extension
6. Scan Engine
7. Dashboard und Analytics
8. Reports und Share Links
9. Open in BC
10. Landingpage und Public Site
11. Admin
12. Security und Datenschutz
13. AppSource und Marketplace
14. Monitoring und Mail Reports
15. Pilot Go-Live Freigabe
16. Customer Go-Live Freigabe
17. AppSource Freigabe
18. Finale Freigabeseite

Datum: 2026-06-10

Status: FINAL ACCEPTANCE CHECKLIST / MANUAL EXECUTION REQUIRED

Diese Datei ist die zentrale Abnahme- und Freigabecheckliste fuer BCSentinel / BC 365 Data Health Management. Sie ersetzt die fruehere manuelle Rohcheckliste und buendelt alle Go-Live-relevanten Pruefungen nach Risiko.

Prioritaeten:

- BLOCKER: Verhindert Pilot oder produktiven Start.
- CRITICAL: Muss vor dem ersten zahlenden Kunden bestanden sein.
- IMPORTANT: Sollte vor breiterem Rollout bestanden sein.
- NICE TO HAVE: Kann nach handgefuehrtem Pilot erfolgen.

Ergebniswerte pro Check: PASS / FAIL / BLOCKED / NOT RUN.

Management Summary

Aktueller Stand aus Code- und Dokumentenpruefung:

- DEV Checkout funktioniert wieder nach Entfernung des Legacy-Imports `PRODUCT_LEGACY_PREMIUM`.
- DEV API muss nach jedem Deployment lokal und oeffentlich per Healthcheck geprueft werden.
- Billing ist aus Codesicht weitgehend vorbereitet, braucht aber dokumentierte Stripe-Testmode-Webhooks fuer alle vier Produkte.
- Landingpage, Header/Footer, Pricing und Admin Translations sind vorbereitet, brauchen Live-Browser-Smoke.
- BC Extension, Scan E2E, Dashboard, Reports und AppSource Analyzer brauchen zwingend manuelle Runtime- bzw. Toolchain-Abnahme.
- PROD Dry Run, Backup und Restore sind fuer zahlende Kunden weiterhin Pflicht.

Aktueller Projektstatus

| Bereich | Prozent | Status | Blocker | Naechster Schritt |

|---|---:|---|---|---

| Infrastruktur | 78% | partial | DEV/PROD Health muss je Deployment dokumentiert werden | DEV Health und nginx Routing nach jedem Deploy pruefen |

| Deployment | 72% | partial | PROD Dry Run und Backup/Restore nicht bestanden dokumentiert | PROD Dry Run mit Migration, Health, Backup, Restore ausfuehren |

| Billing | 80% | partial | Live Webhook- und License-Auswirkungen je Produkt nicht vollstaendig dokumentiert | Stripe Testmode Matrix abarbeiten |

| BC Extension | 74% | partial | AL Compile und aktueller App-Test fehlen | Aktuelle .app bauen, installieren, Setup/Checkout/Scan pruefen |

| Scan Engine | 76% | partial | Deep Scan E2E und Credit-Verbrauch muessen live bestaetigt werden | Quick Scan und Deep Scan aus BC testen |

| Dashboard | 72% | partial | Embed/Monitoring-only Darstellung muss live bestaetigt werden | Dashboard mit One-time und Monitoring Tenant pruefen |

| Reports | 74% | partial | PDF/Share Link/Ablaufzeit live nicht bestaetigt | Executive HTML/PDF und Share Links testen |

| Landingpage | 82% | partial | Live Mobile/DE/EN/Links nicht vollstaendig abgenommen | Public Seiten im Browser pruefen |

| Admin | 80% | partial | Audit-Abdeckung fuer alle kritischen Admin-Aktionen live pruefen | Pricing, Translations, License Actions auditieren |

| Monitoring | 58% | partial | Scheduler/Automation und Monitoring-only UX nicht final abgenommen | Monitoring Tenant E2E pruefen |

| Mail Reports | 30% | missing/unknown | Automatischer Mailversand nicht go-live-bestaetigt | Separaten Mail-Report-Block planen |

Infrastruktur und Deployment

01

DEV Backend und Public Health

BLOCKER

Ziel: Sicherstellen, dass DEV Backend lokal und oeffentlich erreichbar ist.

Voraussetzung: Aktueller staging-Stand ist deployed; .env.dev ist gesetzt.

Schritte:

1. docker compose --env-file .env.dev -f docker-compose.dev.yml up -d --build backend
2. docker compose --env-file .env.dev -f docker-compose.dev.yml ps
3. curl -i http://127.0.0.1:8001/health
4. curl -I https://dev-api.bcsentinel.com/health

Erwartetes Ergebnis: Backend ist Up; lokale und oeffentliche Healthchecks liefern HTTP 200.

PASS-Kriterium: Kein Restart-Loop, kein HTTP 502/503.

FAIL-Kriterium: Backend Restarting, lokaler Healthcheck fehlschlaegt oder Public Health liefert 502/503.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Staging-Stand 8704f9a deployed. Backend wurde mit docker compose neu gebaut und gestartet. Container dhm-dev-backend läuft ohne Restart-Loop. Lokaler Healthcheck <http://127.0.0.1:8001/health> liefert HTTP 200. Öffentlicher Healthcheck <https://dev-api.bcsentinel.com/health> liefert HTTP 200 über nginx. Backend-Logs zeigen erfolgreichen Startup sowie erfolgreiche /health und /health/ready Requests. Kein 502/503 festgestellt.

02

DEV nginx Routing fuer API und Landingpage

BLOCKER

Ziel: DEV API und Landingpage werden an die richtigen Upstreams ausgeliefert.

Voraussetzung: nginx-Konfiguration ist auf dem Server aktiv.

Schritte:

1. sudo nginx -T | grep -nE "server_name dev-api.bcsentinel.com|proxy_pass http://127.0.0.1:8001"
2. sudo nginx -t
3. sudo systemctl reload nginx
4. curl -I https://dev.bcsentinel.com/docs
5. curl -I https://dev.bcsentinel.com/docs.html

Erwartetes Ergebnis: dev-api.bcsentinel.com proxyt auf 127.0.0.1:8001; /docs und /docs.html liefern HTML 200.

PASS-Kriterium: Kein Backend-404 fuer Landingpage-Unterseiten; kein 502 fuer DEV API.

FAIL-Kriterium: Falscher Upstream, Swagger auf dev.bcsentinel.com/docs, 404 oder 502.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

nginx-Konfiguration geprüft: Syntax OK und Reload erfolgreich. dev-api.bcsentinel.com ist auf Backend-Upstream 127.0.0.1:8001 geroutet. dev.bcsentinel.com/docs liefert HTTP 200 text/html, dev.bcsentinel.com/docs.html liefert HTTP 200 text/html. Öffentlicher DEV API Healthcheck <https://dev-api.bcsentinel.com/health> liefert HTTP 200. Kein Backend-404, kein 502/503 festgestellt.

03

Alembic Migration auf Head**BLOCKER**

Ziel: DB Schema entspricht aktuellem Code.

Voraussetzung: Datenbank-Backup oder DEV-Testdatenbank vorhanden.

Schritte:

1. `docker compose --env-file .env.dev -f docker-compose.dev.yml run --rm backend alembic upgrade head`
2. `alembic_version` in der Datenbank prüfen.
3. Backend neu starten und `/health/ready` prüfen.

Erwartetes Ergebnis: Migration läuft ohne Fehler; Readiness ist grün.

PASS-Kriterium: DB ist auf Alembic Head; keine harte alte Revision blockiert Startup.

FAIL-Kriterium: Migration bricht ab, Schema Drift, Startup-Fehler.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Alembic Migration auf Head erfolgreich ausgeführt. Alembic meldet PostgresImpl und startet ohne Fehler. Readiness-Healthcheck auf <http://127.0.0.1:8001/health/ready> liefert HTTP 200 mit Datenbankstatus OK. Backend-Logs zeigen kontinuierlich erfolgreiche Readiness-Checks ohne Exceptions, Startup-Fehler oder Schema-Probleme. Datenbankschema entspricht dem aktuellen Code-Stand.

04

PROD ENV Vollstaendigkeit und Trennung**CRITICAL**

Ziel: PROD startet ohne DEV-Defaults, Test-Keys oder unsichere CORS-Fallbacks.

Voraussetzung: `.env.prod` liegt auf dem Server vor.

Schritte:

1. Pflichtvariablen ohne Secret-Werte prüfen.
2. `APP_BASE_URL`, `CORS_ALLOW_ORIGINS`, `DB`, `SECRET_KEY`, Stripe Live Keys, Billing URLs und `SITE_TRANSLATIONS_PATH` prüfen.
3. Sicherstellen, dass keine DEV-Origins implizit erlaubt sind.

Erwartetes Ergebnis: PROD ist vollstaendig und sicher konfiguriert.

PASS-Kriterium: Keine Test-/DEV-Werte in PROD, keine leeren kritischen Werte.

FAIL-Kriterium: Fehlende Pflichtvariable, Test-Stripe-Key, DEV-Origin in PROD-CORS.

Screenshot erforderlich: Nein

Log erforderlich: Nein

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

PROD läuft auf main Commit 5e94fbd. `.env.prod` wurde um `APP_BASE_URL` sowie Billing Success/Cancel/Portal URLs ergänzt. Backend wurde mit aktualisierter PROD-ENV neu erstellt. Dabei wurden mehrere Go-Live-relevante Findings behoben: alte Runtime-ENV im Backend-Container, DB-Passwort/Runtime-Abgleich und ausstehende Alembic-Migrationen. Vor dem Alembic-Fix wurde ein DB-Backup erstellt. `alembic_version.version_num` wurde von `varchar(32)` auf `varchar(255)` erweitert; Migrationen liefen anschließend bis 0013_scan_stability_layer durch. Backend startet erfolgreich, Postgres ist healthy. Lokaler `/health` liefert HTTP 200, lokaler `/health/ready` liefert HTTP 200 mit `database=ok`, öffentlicher Healthcheck <https://api.bcsentinel.com/health> liefert HTTP 200. Kein 502/503 mehr.

05

PROD Dry Run**CRITICAL**

Ziel: PROD Deployment ist reproduzierbar startfaehig.

Voraussetzung: Wartungsfenster oder isolierte PROD-Dry-Run-Umgebung.

Schritte:

1. docker compose --env-file .env.prod -f docker-compose.prod.yml build
2. docker compose --env-file .env.prod -f docker-compose.prod.yml run --rm migration
3. docker compose --env-file .env.prod -f docker-compose.prod.yml up -d backend
4. sudo nginx -t && sudo systemctl reload nginx
5. curl -i http://127.0.0.1:8000/health/ready
6. curl -I https://api.bcsentinel.com/health

Erwartetes Ergebnis: Build, Migration, Backend, nginx und Healthchecks laufen durch.

PASS-Kriterium: Keine 500/502/503, kein Container-Restart-Loop.

FAIL-Kriterium: Migration oder Healthcheck scheitert.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

PROD Docker Build erfolgreich durchgeführt. Backend Container erfolgreich erstellt und gestartet. PostgreSQL Container healthy. Lokaler Healthcheck /health liefert HTTP 200. Lokaler Readiness-Check /health/ready liefert HTTP 200 mit Datenbankstatus OK. Öffentlicher Healthcheck https://api.bcsentinel.com/health liefert HTTP 200. Öffentlicher Readiness-Check https://api.bcsentinel.com/health/ready liefert HTTP 200. nginx-Konfiguration erfolgreich validiert. PROD Alembic-Version bestätigt: 0013_scan_stability_layer. Abweichung dokumentiert: In docker-compose.prod.yml existiert kein separater Service migration. Die Migration wurde bereits erfolgreich im Check 04-prod-env durchgeführt und verifiziert. Keine Container-Restart-Loops festgestellt. Keine HTTP 500, 502 oder 503 nach erfolgreichem Recheck.

06

Backup und Restore**BLOCKER**

Ziel: Daten koennen vor Go-Live gesichert und wiederhergestellt werden.

Voraussetzung: Zugriff auf PROD- oder Restore-Testdatenbank.

Schritte:

1. pg_dump "\$DATABASE_URL" > backup-before-golive.sql
2. psql "\$RESTORE_TEST_DATABASE_URL" < backup-before-golive.sql
3. Tabellen und alembic_version in Restore-DB pruefen.

Erwartetes Ergebnis: Backup und Restore laufen ohne Fehler.

PASS-Kriterium: Restore-DB ist lesbar und schema-kompatibel.

FAIL-Kriterium: Backup fehlt, Restore bricht ab oder Datenbank unbrauchbar.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Separate Restore-Testdatenbank dhm_restore_test erstellt. Produktionsdatenbank dhm blieb unverändert. Backup der PROD-Datenbank erfolgreich erstellt. Restore in dhm_restore_test erfolgreich durchgeführt. Restore ohne Fehler abgeschlossen (Exit Code 0). Alembic-Version nach Restore erfolgreich geprüft: 0013_scan_stability_layer. Restore-Datenbank enthält 20 Tabellen im Schema public. Backup- und Restore-Prozess vollständig praktisch validiert. Nachweis für Disaster-Recovery und Wiederherstellbarkeit erbracht. Keine Datenverluste festgestellt. Kein Einsatz von docker compose down -v.

Ziel: Fehlerhafte Deployments koennen ohne Datenverlust zurueckgenommen werden.

Voraussetzung: Vorheriger stabiler Release/Commit ist bekannt.

Schritte:

1. Rollback-Befehl aus docs/DEPLOYMENT_DRY_RUN.md in DEV ausfuehren oder trocken validieren.
2. Healthcheck nach Rollback pruefen.
3. Kein docker compose down -v verwenden.

Erwartetes Ergebnis: System laeuft nach Rollback wieder.

PASS-Kriterium: Health gruen, keine geloeschten Volumes.

FAIL-Kriterium: Rollback unklar, Datenverlust oder Health rot.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Rollback-Prozess wurde trocken validiert. DEPLOYMENT_DRY_RUN.md lag in DEV vor und wurde in den PROD-Checkout kopiert. Rollback-Abschnitt vorhanden. Hinweis gegen docker compose down -v ist dokumentiert. Backend und PostgreSQL liefen stabil. Lokale und öffentliche Health-/Readiness-Checks lieferten HTTP 200. Rollback wurde bewusst nicht real ausgeführt. Datei ist aktuell noch untracked und muss ins Repository committed werden.

Billing und Stripe

Ziel: Alle vier Produktpreise sind zentral sichtbar, editierbar und oeffentlich abrufbar.

Voraussetzung: Admin Login erfolgreich; Backend erreichbar.

Schritte:

1. /admin/config/license-pricing oeffnen.
2. Produkte assessment, validation_check, monitoring_monthly, monitoring_annual pruefen.
3. DEV-Testaenderung speichern und wieder rueckgaengig machen.
4. /pricing/public pruefen.
5. Admin Audit pruefen.

Erwartetes Ergebnis: Preise 79, 49, 99/Monat und 990/Jahr sind konsistent.

PASS-Kriterium: UI, Public API und Audit stimmen.

FAIL-Kriterium: Produkt fehlt, Preis wird nicht gespeichert, Public API falsch.

Screenshot erforderlich: Ja

Log erforderlich: Nein

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

PROD wurde auf DEV/Staging-Stand 8704f9a migriert. Alembic erfolgreich von 0013_scan_stability_layer auf 0017_tenant_preferred_language aktualisiert. Fehlende PROD-ENV-Werte CORS_ALLOW_ORIGINS und TENANT_REGISTRATION_INVITE_CODE ergänzt. Backend danach healthy. /pricing/public liefert HTTP 200 und die vier neuen Produkte korrekt aus: Assessment 79 €, Validation Check 49 €, Monitoring Monthly 99 €/Monat, Monitoring Annual 990 €/Jahr. Tabelle product_pricing_config enthält alle vier Produkte aktiv und konsistent.

Admin Basic Authentication wurde erfolgreich validiert. Unauthentifizierte Zugriffe liefern HTTP 401. Nach Ergänzung der neuen PROD-Konfigurationswerte (CORS_ALLOW_ORIGINS und TENANT_REGISTRATION_INVITE_CODE) konnte das Admin-Panel erfolgreich geöffnet werden. Product Pricing ist erreichbar und basiert auf dem neuen Product-Licensing-Modell.

Ziel: One-time Checkout fuer Assessment funktioniert.

Voraussetzung: STRIPE_PRICE_ID_ASSESSMENT ist gesetzt; Stripe Testmode aktiv.

Schritte:

1. Checkout aus BC oder Dashboard starten.
2. Stripe-Testzahlung erfolgreich abschliessen.
3. Success-Seite pruefen.
4. Webhook und License Status pruefen.

Erwartetes Ergebnis: Assessment-Testartikel, Purchase, 7 Tage Zugriff und +1 Scan Credit.

PASS-Kriterium: Kein 503/500, Webhook verarbeitet, Credit sichtbar.

FAIL-Kriterium: Falscher Artikel, kein Webhook, kein Credit.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Assessment Checkout wurde erfolgreich für Tenant ten_aa37d035f0a6 erzeugt. Erste unauthentifizierte Anfrage wurde korrekt mit 401 abgelehnt. Mit Tenant-Header und Body-tenant_id wurde eine Stripe Checkout Session erfolgreich erstellt. Stripe Webhook wurde von DEV auf PROD korrigiert (<https://api.bcsentinel.com/billing/webhook>). Nach erneutem Webhook-Delivery kam checkout.session.completed mit HTTP 200 an. Purchase wurde angelegt: product_code=assessment, provider=stripe, status=paid, amount_total=79. Ein Scan-Credit wurde erzeugt: product_code=assessment, status=available, source=checkout, source_purchase_id=1. Stripe Event-ID dokumentiert: evt_1ThZo0A4LGkXkz25GjDHvIh.

Ziel: One-time Checkout fuer Validation Check funktioniert.

Voraussetzung: STRIPE_PRICE_ID_VALIDATION_CHECK ist gesetzt; Stripe Testmode aktiv.

Schritte:

1. Checkout starten.
2. Stripe-Testzahlung erfolgreich abschliessen.
3. Webhook, License Status und Credit pruefen.

Erwartetes Ergebnis: Validation-Testartikel, Purchase, 7 Tage Zugriff und +1 Scan Credit.

PASS-Kriterium: License API und BC Setup zeigen korrekten Zugriff.

FAIL-Kriterium: Falsches Produkt, kein Credit, Webhookfehler.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Billing Validation Check (PROD) erfolgreich durchgeführt. Nach erfolgreicher Registrierung der BC Extension gegen die Produktivumgebung wurde der Lizenzstatus erfolgreich abgerufen und ein Validation Check Kauf direkt aus Microsoft Dynamics 365 Business Central gestartet. Die Stripe Checkout Session wurde erfolgreich erstellt und die Testzahlung vollständig abgeschlossen. Das Stripe Event checkout.session.completed wurde vom Webhook-Endpoint /billing/webhook erfolgreich verarbeitet. Anschließend wurde automatisch ein Kaufdatensatz mit dem Produkt validation_check, dem Provider stripe, dem Status paid und dem Betrag von 49 EUR in der Datenbank angelegt. Gleichzeitig wurde dem Tenant ein verfügbarer Validation-Check-Credit mit Status available und Quelle checkout gutgeschrieben. Die anschließende Lizenzabfrage gegen /license/status lieferte erfolgreich 200 OK zurück. Damit wurden Checkout-Erstellung, Zahlungsabwicklung, Webhook-Verarbeitung, Kaufverbuchung, Credit-Erstellung und Lizenzaktualisierung in der Produktivumgebung vollständig und erfolgreich validiert.

12

Monitoring Monthly Kauf**BLOCKER**

Ziel: Monatlicher Subscription Checkout funktioniert.

Voraussetzung: STRIPE_PRICE_ID_MONITORING_MONTHLY ist gesetzt und recurring monthly.

Schritte:

1. Checkout starten.
2. Stripe-Testzahlung erfolgreich abschliessen.
3. Webhook, Subscription, License Status und Dashboard Access pruefen.

Erwartetes Ergebnis: Subscription ist aktiv, monitoring_active=true.

PASS-Kriterium: Monitoring-Zugriff aktiv und im Admin sichtbar.

FAIL-Kriterium: Payment Mode statt Subscription, Monitoring bleibt inaktiv.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Monitoring Monthly Kauf (PROD) erfolgreich durchgeführt. Der Kauf des monatlichen Monitorings wurde aus der BC Extension gestartet und über Stripe als Subscription abgeschlossen. Die Checkout Session wurde erfolgreich erstellt und verarbeitet. Die Stripe Webhooks checkout.session.completed, subscription.created und subscription.updated wurden erfolgreich verarbeitet. In der Datenbank wurde eine aktive Subscription mit dem Tarif monitoring_monthly und einem monatlichen Betrag von 99 EUR angelegt. Der Lizenzstatus konnte anschließend erfolgreich aktualisiert werden. In der BC Extension wurde der Produktzugriff auf monitoring umgestellt, der Funktionszugriff zeigte Monitoring active und Deep Scans waren freigegeben. Damit wurden Subscription-Erstellung, Webhook-Verarbeitung, Lizenzaktivierung und Monitoring-Freischaltung erfolgreich validiert.

13

Monitoring Annual Kauf**BLOCKER**

Ziel: Jaehrlicher Subscription Checkout funktioniert.

Voraussetzung: STRIPE_PRICE_ID_MONITORING_ANNUAL ist gesetzt und recurring yearly.

Schritte:

1. Checkout starten.
2. Stripe-Testzahlung erfolgreich abschliessen.
3. Webhook, Subscription, License Status und Dashboard Access pruefen.

Erwartetes Ergebnis: Annual Subscription ist aktiv, monitoring_active=true.

PASS-Kriterium: Richtiger Jahresartikel, Monitoring aktiv.

FAIL-Kriterium: Falscher Intervall oder kein Monitoring Access.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Monitoring Annual (PROD) erfolgreich mit neuem Test-Tenant (ten_1b9b492d58e9) durchgeführt. Stripe Checkout erfolgreich erstellt und abgeschlossen. Webhooks checkout.session.completed, subscription.created und subscription.updated erfolgreich verarbeitet. Subscription sub_1ThcYKA4LGkXkz2d592Dy3L mit Status active und plan_code monitoring_annual angelegt. Monitoring und Dashboard-Zugriff automatisch aktiviert. Bestehender Monthly-Tenant blieb unverändert aktiv. Während des Tests wurden Legacy-Stripe-Preisreferenzen, ein Webhook-Reihenfolgeproblem sowie ein Deployment-/Alembic-Problem identifiziert und behoben. Ergebnis: PASS WITH NOTE. Offener Punkt: current_period_end_utc wird aktuell nicht befüllt und daher kein Laufzeitende angezeigt.

14

Stripe Success und Cancel Flow

CRITICAL

Ziel: Erfolgreiche und abgebrochene Checkouts sind sauber dargestellt.

Voraussetzung: Stripe Checkout kann geöffnet werden.

Schritte:

1. Einen Checkout erfolgreich abschliessen.
2. Einen Checkout abbrechen.
3. `billing-success.html` und `billing-cancel.html` in DE/EN prüfen.

Erwartetes Ergebnis: Success bestätigt Kauf, Cancel erzeugt keine Lizenz-/Credit-Änderung.

PASS-Kriterium: Keine Legacy-Begriffe, keine i18n Keys, kein Mojibake.

FAIL-Kriterium: Falsche Produkttexte, Credit trotz Abbruch, kaputte Seite.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

15

Stripe Webhook und Event-ID Dokumentation

BLOCKER

Ziel: Stripe Events werden signiert empfangen, verarbeitet und dokumentiert.

Voraussetzung: Stripe Webhook zeigt auf `/billing/webhook`.

Schritte:

1. Pro Produkt `checkout.session.completed` auslösen.
2. Backend-Logs prüfen.
3. Stripe Event ID je Test notieren.
4. Duplicate-Verarbeitung grob prüfen.

Erwartetes Ergebnis: Events werden ohne Signatur- oder Duplicatefehler verarbeitet.

PASS-Kriterium: Event ID dokumentiert, DB-/License-Auswirkungen korrekt.

FAIL-Kriterium: Kein Event, Signaturfehler, 400/500, doppelte Credits.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

16

Customer Portal und Subscription Cancel

CRITICAL

Ziel: Monitoring-Subscriptions koennen verwaltet und gekuendigt werden.

Voraussetzung: Aktive Monitoring-Testsubscription.

Schritte:

1. Customer Portal starten.
2. Subscription kuendigen.
3. Webhook und License Status pruefen.

Erwartetes Ergebnis: Portal oeffnet, Kuendigung wird verarbeitet, Status passt.

PASS-Kriterium: Monitoring-Zugriff endet gemaess Stripe-/Produktlogik.

FAIL-Kriterium: Portalfehler, Status bleibt falsch, Webhookfehler.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Business Central Extension

17

AL Package Build und aktuelle App-Version

BLOCKER

Ziel: Die getestete BC Extension entspricht dem aktuellen Code.

Voraussetzung: AL Toolchain und BC Sandbox vorhanden.

Schritte:

1. AL Compile/Package ausfuehren.
2. .app in BC Sandbox veroeffentlichen.
3. App-Version in Extension Management mit app.json vergleichen.

Erwartetes Ergebnis: Build und Installation laufen ohne Fehler.

PASS-Kriterium: Aktuelle Version ist installiert und aktiv.

FAIL-Kriterium: Compile-/Publish-/Upgradefehler oder alte Version aktiv.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Ziel: Marketplace-/Pilotkunde kann Setup ohne Entwicklerhilfe starten.

Voraussetzung: App installiert; DEV API erreichbar.

Schritte:

1. BCSentinel Setup Page oeffnen.
2. API URL auf <https://dev-api.bcsentinel.com> setzen.
3. Connection Test ausfuehren.
4. Tenant registrieren.
5. API Token Status pruefen.

Erwartetes Ergebnis: Tenant wird im Backend angelegt; Token ist konfiguriert, aber nicht sichtbar.

PASS-Kriterium: Setup, Connection Test und Registrierung erfolgreich.

FAIL-Kriterium: 401/500/502, Token sichtbar, Tenant fehlt.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Ziel: BC zeigt Credits, Zugriff und Monitoring korrekt.

Voraussetzung: Tenant registriert; mindestens ein Produkt gekauft oder manuell vergeben.

Schritte:

1. License Refresh ausfuehren.
2. Scan Credits, Access Windows und Monitoring Status pruefen.
3. DE/EN Texte in BC pruefen.

Erwartetes Ergebnis: Werte entsprechen Backend License Status.

PASS-Kriterium: Credits und Monitoring stimmen; keine Rohfehler.

FAIL-Kriterium: Falsche Werte, Sprachmix, technische Fehlermeldungen.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Ziel: Alle vier BC-Kaufaktionen starten den richtigen Stripe Checkout.

Voraussetzung: Billing-Konfiguration und API URL korrekt.

Schritte:

1. Assessment aus BC starten.
2. Validation Check aus BC starten.
3. Monitoring Monthly aus BC starten.
4. Monitoring Annual aus BC starten.

Erwartetes Ergebnis: Jeweils richtiger Stripe-Testartikel.

PASS-Kriterium: Kein 503, kein falsches Produkt, keine Legacy-Angebote.

FAIL-Kriterium: Fehler, falscher Artikel, alter Premium/Trial/Free Text.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
------------------	------	------	---------	---------

Notizen:

Scan Engine

Quick Scan E2E

IMPORTANT

Ziel: Quick Scan funktioniert aus BC bis Backend und Ergebnis.

Voraussetzung: Tenant registriert; API Token gueltig.

Schritte:

1. Quick Scan in BC starten.
2. Backend-Request und Ergebnis prüfen.
3. Anzeige und Sprache prüfen.

Erwartetes Ergebnis: Scan laeuft ohne Fehler und Ergebnis ist sichtbar.

PASS-Kriterium: Ergebnis plausibel, keine Rohfehler.

FAIL-Kriterium: Request fehlt, 500/502, leeres Ergebnis.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
------------------	------	------	---------	---------

Notizen:

Deep Scan E2E und Scan History

BLOCKER

Ziel: Deep Scan startet wirklich, erzeugt History und verbraucht Credits korrekt.

Voraussetzung: One-time Credit oder Monitoring Access vorhanden.

Schritte:

1. Deep Scan bestaetigen.
2. Backend-Start-Endpunkt im Log pruefen.
3. Status queued/running/completed im Monitor pruefen.
4. Scan History oeffnen.
5. Credit vor/nach Scan vergleichen.

Erwartetes Ergebnis: Neuer Run, History-Eintrag, korrekter Status und genau ein Credit-Verbrauch bei One-time.

PASS-Kriterium: Kein Fake-Erfolg, kein doppelter Verbrauch.

FAIL-Kriterium: Nur Status-Update ohne Start, kein History-Eintrag, falscher Credit-Verbrauch.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Monitoring Scan ohne Credit

CRITICAL

Ziel: Monitoring-Tenants koennen gemaess Produktlogik wiederholt scannen.

Voraussetzung: Monitoring Monthly oder Annual aktiv; Scan Credits optional 0.

Schritte:

1. License Status auf Monitoring aktiv pruefen.
2. Deep Scan ohne verfuegbaren One-time Credit starten.
3. History und Status pruefen.

Erwartetes Ergebnis: Scan startet wegen Monitoring Access.

PASS-Kriterium: Kein faelschlicher Credit-Blocker.

FAIL-Kriterium: Scan wird trotz Monitoring blockiert.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Scan Fehlerfall

CRITICAL

Ziel: Fehler werden sichtbar, sicher und ohne Token-Leaks gemeldet.

Voraussetzung: Kontrollierter Fehlerfall moeglich.

Schritte:

1. Fehlerfall ausloesen, z. B. ungultige API URL oder fehlende Berechtigung.
2. BC Meldung und Backend-Log pruefen.

Erwartetes Ergebnis: Sichere Nutzerfehlermeldung mit Supporthinweis/Request-ID.

PASS-Kriterium: Keine Tokens, Header, Stacktraces oder internen URLs in UI.

FAIL-Kriterium: Rohes JSON, Token, Stacktrace oder sensible Details.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Dashboard und Analytics

Dashboard aus BC oeffnen

BLOCKER

Ziel: BC oeffnet das richtige Tenant-Dashboard.

Voraussetzung: Tenant registriert; Dashboard Access aktiv.

Schritte:

1. Dashboard Aktion in BC ausfuehren.
2. Rendering im Browser/ControlAddIn pruefen.
3. Data Health Score und Issue Details pruefen.

Erwartetes Ergebnis: Dashboard laedt ohne Tokenanzeige und mit korrekten Tenantdaten.

PASS-Kriterium: Kein sichtbarer Token, kein 401/500, Werte plausibel.

FAIL-Kriterium: Leeres Dashboard, falscher Tenant, Token sichtbar.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Dashboard Sprache DE/EN

IMPORTANT

Ziel: Dashboard folgt BC-/Tenant-Sprache.

Voraussetzung: DE- und EN-Testtenant.

Schritte:

1. Dashboard fuer deutschen Tenant oeffnen.
2. Dashboard fuer nicht-deutschen Tenant oeffnen.
3. Labels, Severity, Module, Empty States und Buttons pruefen.

Erwartetes Ergebnis: Deutsch fuer deutsche BC-Sprache, sonst Englisch.

PASS-Kriterium: Keine sichtbare Sprachmischung oder fehlende Keys.

FAIL-Kriterium: undefined, i18n Keys oder DE/EN-Mix.

Screenshot erforderlich: Ja

Log erforderlich: Nein

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Monitoring-only Dashboard-Bereiche

CRITICAL

Ziel: ScoreTrend, LossTrend und Recent Scans erscheinen nur bei Monitoring.

Voraussetzung: Vergleich von One-time Tenant und Monitoring Tenant.

Schritte:

1. One-time Dashboard oeffnen.
2. Monitoring Dashboard oeffnen.
3. ScoreTrend, LossTrend, Recent Scans und CTAs vergleichen.

Erwartetes Ergebnis: Monitoring-Mehrwert erscheint nur bei Monitoring.

PASS-Kriterium: Keine falsche Subscription-/Trendanzeige bei Assessment/Validation.

FAIL-Kriterium: Monitoring-Bereiche bei Nicht-Monitoring sichtbar oder umgekehrt.

Screenshot erforderlich: Ja

Log erforderlich: Nein

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Analytics Embed Token Sicherheit

CRITICAL

Ziel: Keine API Tokens in URLs, UI oder Logs.

Voraussetzung: Dashboard wurde geöffnet.

Schritte:

1. Browser URL und Network grob prüfen.
2. Backend-Logs nach Tokens/Headers durchsuchen.
3. ControlAddIn Lade-/Fehlerzustand prüfen.

Erwartetes Ergebnis: Kein API Token sichtbar oder geloggt.

PASS-Kriterium: Nur kurzlebiger Embed-/Session-Mechanismus, kein API Token.

FAIL-Kriterium: API Token in Querystring, UI, Console oder Log.

Screenshot erforderlich: Nein

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
------------------	------	------	---------	---------

Notizen:

Reports und Share Links

Executive Report HTML und PDF

BLOCKER

Ziel: Executive Reports sind fuer Pilotkunden professionell nutzbar.

Voraussetzung: Abgeschlossener Scan mit Findings.

Schritte:

1. Executive HTML oeffnen.
2. Executive PDF oeffnen und herunterladen.
3. Management Summary, Risk/Priority Matrix und Next Best Action prüfen.
4. DE/EN mit Testtenants prüfen.

Erwartetes Ergebnis: HTML/PDF sind lesbar, professionell und sprachlich konsistent.

PASS-Kriterium: Keine Platzhalter, keine Mojibake, PDF oeffnet korrekt.

FAIL-Kriterium: 500, leeres PDF, fehlende Kernaussagen, Sprachmix.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
------------------	------	------	---------	---------

Notizen:

Report Share Links HTML/PDF

BLOCKER

Ziel: Share Links funktionieren sicher und ohne Login im erlaubten Zeitraum.

Voraussetzung: Share Link fuer HTML und PDF erzeugt.

Schritte:

1. HTML Share Link oeffnen.
2. PDF Share Link oeffnen.
3. Beide Links im privaten Browser oeffnen.
4. Tenantdaten und Ablaufhinweis pruefen.

Erwartetes Ergebnis: Links funktionieren bis Ablauf und zeigen nur den richtigen Report.

PASS-Kriterium: Kein Login erforderlich, keine fremden Daten, kein 500.

FAIL-Kriterium: Falscher Tenant, Session-Abhaengigkeit, kaputter Link.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Share Link Ablaufzeit und Base URL

BLOCKER

Ziel: Share Links laufen ab und enthalten keine lokalen URLs.

Voraussetzung: APP_BASE_URL ist gesetzt; Testlink mit bekannter TTL.

Schritte:

1. Link vor Ablauf pruefen.
2. Link nach Ablauf pruefen.
3. Linktext/URL auf localhost und 127.0.0.1 pruefen.

Erwartetes Ergebnis: Vor Ablauf 200, nach Ablauf sichere Sperre, oeffentliche Base URL.

PASS-Kriterium: Kein localhost, Ablauf greift.

FAIL-Kriterium: Link bleibt unbegrenzt aktiv oder enthaelt localhost.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Open in BC

Open in BC Issue-Deep-Link

IMPORTANT

Ziel: Dashboard-Issue fuehrt direkt zur relevanten BC-Seite.

Voraussetzung: Dashboard mit Issue und BC Session.

Schritte:

1. Issue Detail im Dashboard oeffnen.
2. Open in BC klicken.
3. Zielseite, Tenant, Company und Datensatz pruefen.

Erwartetes Ergebnis: Nutzer landet direkt auf passender BC-Seite ohne unnoetige Drilldown-Zwischenseite.

PASS-Kriterium: Kontext stimmt, kein Cross-Tenant/Company-Fehler.

FAIL-Kriterium: Falsche Seite, falscher Tenant, generische Zwischenseite.

Screenshot erforderlich: Ja

Log erforderlich: Nein

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Open in BC Fehlerfall

IMPORTANT

Ziel: Fehlende BC-Berechtigungen werden sicher behandelt.

Voraussetzung: Testnutzer mit eingeschaenkten Rechten.

Schritte:

1. Open in BC mit eingeschaenktem Nutzer ausfuehren.
2. BC Meldung pruefen.

Erwartetes Ergebnis: Verstaendliche Fehlermeldung ohne Datenleck.

PASS-Kriterium: Kein fremder Zugriff, keine technischen Rohdetails.

FAIL-Kriterium: Datenleck oder unklare technische Fehlermeldung.

Screenshot erforderlich: Ja

Log erforderlich: Nein

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Landingpage und Public Site

Public Seiten erreichbar

BLOCKER

Ziel: Alle oeffentlichen Seiten liefern HTTP 200.

Voraussetzung: Landingpage deployed.

Schritte:

1. index.html, docs.html, security.html, privacy.html, terms.html, contact.html, impressum.html, loss-examples.html oeffnen.

2. Partner- und Billing-Seiten oeffnen.

3. Linkcheck fuer Header/Footer durchfuehren.

Erwartetes Ergebnis: Keine 404/502, alle Seiten rendern.

PASS-Kriterium: Pflichtseiten erreichbar und Links funktionieren.

FAIL-Kriterium: Seite fehlt, Link tot, Backend-404.

Screenshot erforderlich: Ja

Log erforderlich: Nein

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Header/Footer und Navigation

IMPORTANT

Ziel: Einheitlicher Site-Shell-Look mit seitenspezifischer Header-Navigation und globalem Footer.

Voraussetzung: site-shell.js aktiv.

Schritte:

1. Alle Pflichtseiten im Desktop und Mobile Viewport pruefen.

2. Header-Menuepunkte auf vorhandene Section-IDs pruefen.

3. Footer-Links als echte Seitenlinks pruefen.

Erwartetes Ergebnis: Einheitlicher Look, keine zusammengeklebten Links, keine toten Scroll-Ziele.

PASS-Kriterium: Header/Footer konsistent und funktional.

FAIL-Kriterium: Alte Header-Markups, tote Links, kaputte mobile Navigation.

Screenshot erforderlich: Ja

Log erforderlich: Nein

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Landingpage DE/EN und Theme Toggle

IMPORTANT

Ziel: Sprache und Theme funktionieren auf allen Seiten.

Voraussetzung: landingpage/lang/de.json und en.json erreichbar.

Schritte:

1. DE/EN umschalten.
2. Reload und localStorage Persistenz pruefen.
3. Light/Dark toggeln.
4. Desktop und Mobile pruefen.

Erwartetes Ergebnis: Texte wechseln, Theme bleibt gespeichert, kein sichtbarer Text Light/Dark.

PASS-Kriterium: Keine i18n Keys, kein undefined, keine Theme-Textfehler.

FAIL-Kriterium: Toggle ohne Wirkung, fehlende Texte, sichtbare Keys.

Screenshot erforderlich: Ja

Log erforderlich: Nein

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Landingpage Pricing und Produkttexte

BLOCKER

Ziel: Public Site zeigt nur neue Produktstruktur und zentrale Preise.

Voraussetzung: /pricing/public erreichbar.

Schritte:

1. Pricing Cards mit Admin Product Pricing vergleichen.
2. Browser-Suche nach Free, Trial, Premium durchfuehren.
3. Preise 79, 49, 99/Monat und 990/Jahr pruefen.

Erwartetes Ergebnis: Keine alten Kundentexte oder falschen Preise.

PASS-Kriterium: Assessment, Validation Check, Monitoring Monthly, Monitoring Annual korrekt.

FAIL-Kriterium: Alte Free/Trial/Premium-Reste sichtbar oder Preis falsch.

Screenshot erforderlich: Ja

Log erforderlich: Nein

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Public Site Encoding und Assets

IMPORTANT

Ziel: Keine Mojibake, kaputten Bilder oder Layoutfehler.

Voraussetzung: Public Seiten erreichbar.

Schritte:

1. Seiten visuell pruefen.
2. Besonders Euro-Zeichen, Umlaute, Progress/Charts und Icons pruefen.
3. Browser Console auf Asset-Fehler pruefen.

Erwartetes Ergebnis: Keine Zeichen wie Ã, â, ■; Assets laden.

PASS-Kriterium: Sauberes Rendering.

FAIL-Kriterium: Mojibake, fehlende Assets, JS-Fehler.

Screenshot erforderlich: Ja

Log erforderlich: Nein

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Admin

Admin Login und Zugriffsschutz

BLOCKER

Ziel: Admin ist geschuetzt und Login funktioniert.

Voraussetzung: Admin Credentials liegen vor.

Schritte:

1. /admin ohne Login im privaten Browser oeffnen.
2. Mit Login anmelden.
3. Dashboard und Menue pruefen.

Erwartetes Ergebnis: Ohne Login kein Zugriff; mit Login Admin UI sichtbar.

PASS-Kriterium: Kein Admin-Inhalt ohne Login, keine 500-Seite nach Login.

FAIL-Kriterium: Admin offen oder Login defekt.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Admin Translations

CRITICAL

Ziel: Landingpage- und Unterseiten-Texte sind im Admin vollstaendig bearbeitbar.

Voraussetzung: SITE_TRANSLATIONS_PATH zeigt auf vorhandene de.json und en.json.

Schritte:

1. Admin Translations oeffnen.
2. Seitengruppen und Common / Shared pruefen.
3. Einen DEV-Testtext speichern und rueckgaengig machen.

Erwartetes Ergebnis: Keine FileNotFoundError-Meldung; alle Keys sichtbar.

PASS-Kriterium: Laden und Speichern funktionieren; unbekannte Keys erscheinen unter Common / Shared.

FAIL-Kriterium: 500, fehlende Dateien, fehlende Keys, Speichern defekt.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Admin Audit Log

CRITICAL

Ziel: Kritische Admin-Aktionen sind nachvollziehbar.

Voraussetzung: Pricing-, Translation- oder License-Aktion wurde ausgefuehrt.

Schritte:

1. Admin Audit oeffnen.
2. Benutzer, Zeitpunkt, Aktion und vorher/nachher Details pruefen.

Erwartetes Ergebnis: Relevante Aktion ist auditierbar.

PASS-Kriterium: Wer, wann, was und Details sichtbar.

FAIL-Kriterium: Keine Audit-Zeile oder unbrauchbare Details.

Screenshot erforderlich: Ja

Log erforderlich: Nein

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Tenant- und License-Administration

IMPORTANT

Ziel: Support kann Pilottenant steuern, ohne Daten zu beschadigen.

Voraussetzung: Testtenant vorhanden.

Schritte:

1. Tenant im Admin oeffnen.
2. Manual Grants, Credits und Product Access pruefen.
3. Audit-Eintrag pruefen.

Erwartetes Ergebnis: Admin-Aktionen sind klar, sicher und auditierbar.

PASS-Kriterium: Keine falschen Produktnamen, keine stillen Datenveraenderungen.

FAIL-Kriterium: Unklare Aktionen, falsche Grants, kein Audit.

Screenshot erforderlich: Ja

Log erforderlich: Nein

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Security und Datenschutz

Secret- und Token-Schutz

BLOCKER

Ziel: Keine Secrets im Repo, UI, URL oder Log.

Voraussetzung: Repo Zugriff und ausgefuehrte Testflows.

Schritte:

1. Repo nach echten Keys/Tokens pruefen.
2. Browser URLs und Network fuer Dashboard/Reports/Checkout pruefen.
3. Backend-Logs nach Tokens, Headers und sensiblen Payloads pruefen.

Erwartetes Ergebnis: Keine Secrets oder API Tokens sichtbar.

PASS-Kriterium: Nur Platzhalter im Repo; keine API Tokens in Querystrings/Logs.

FAIL-Kriterium: Live/Test Secret im Repo oder Token-Leak.

Screenshot erforderlich: Nein

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Tenant-Isolation

BLOCKER

Ziel: Tenant A kann keine Daten von Tenant B lesen.

Voraussetzung: Zwei Testtenants mit unterschiedlichen Tokens.

Schritte:

1. Tenant-A Token gegen Tenant-B Ressourcen verwenden.
2. Scan, Dashboard, Report und License APIs pruefen.

Erwartetes Ergebnis: Zugriff wird verweigert.

PASS-Kriterium: 403/404 ohne Datenleck.

FAIL-Kriterium: Cross-Tenant-Daten sichtbar.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

CORS und Security Headers

CRITICAL

Ziel: Browser-Sicherheitskonfiguration ist produktionsfaehig.

Voraussetzung: DEV/PROD Domains erreichbar.

Schritte:

1. API und Landingpage mit curl -I pruefen.
2. CORS Header und Security Header pruefen.
3. PROD ohne implizite DEV-Origin pruefen.

Erwartetes Ergebnis: Saubere Header, keine unsicheren PROD-Fallbacks.

PASS-Kriterium: Keine DEV-Origin in PROD-CORS; Security Header vorhanden.

FAIL-Kriterium: Offene CORS-Konfiguration, fehlende kritische Header.

Screenshot erforderlich: Ja

Log erforderlich: Nein

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Datenschutz- und Consent-Flow

CRITICAL

Ziel: BC Admin versteht Datenuebertragung und Consent ist Pflicht.

Voraussetzung: BC Setup Page erreichbar.

Schritte:

1. Consent-Text in BC Setup pruefen.
2. Registrierung/Scan ohne Consent testen.
3. Privacy/Terms Links pruefen.

Erwartetes Ergebnis: Consent ist verstaendlich und erforderlich.

PASS-Kriterium: Keine Registrierung/Scan ohne erforderlichen Consent.

FAIL-Kriterium: Consent fehlt, ist unklar oder wird nicht erzwungen.

Screenshot erforderlich: Ja

Log erforderlich: Nein

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

AppSource und Marketplace

AL Analyzer

CRITICAL

Ziel: CodeCop, UICop und AppSourceCop sind ausgefuehrt und bewertet.

Voraussetzung: AL Toolchain und Analyzer-Konfiguration.

Schritte:

1. AL Compile mit CodeCop ausfuehren.
2. UICop ausfuehren.
3. AppSourceCop ausfuehren.
4. Warnings/Errors dokumentieren.

Erwartetes Ergebnis: Keine unbekannten technischen AppSource-Blocker.

PASS-Kriterium: Nur bekannte finale Metadaten/ID-Range-Themen offen.

FAIL-Kriterium: Compile Error oder unbewertete AppSourceCop Errors.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

app.json und Marketplace Pflichtmetadaten

CRITICAL

Ziel: Manifest und Listing-Metadaten sind AppSource-nahe.

Voraussetzung: Aktuelle BC Extension Quellen.

Schritte:

1. Publisher, Name, Description, Version, idRanges, runtime und URLs pruefen.
2. Privacy, Terms/EULA, Help, Support und Contact URL pruefen.
3. Logo-Pfad und Datei pruefen.

Erwartetes Ergebnis: Keine Fake-Live-URLs oder undokumentierten Placeholder.

PASS-Kriterium: Finale Werte vorhanden oder als Submission-TODO dokumentiert.

FAIL-Kriterium: Fehlende Pflichtmetadaten ohne TODO.

Screenshot erforderlich: Nein

Log erforderlich: Nein

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

AppSource Assets und Texte

IMPORTANT

Ziel: Listing-Material ist bereit fuer Submission.

Voraussetzung: Finale Produktpositionierung und Screenshots.

Schritte:

1. Kurzbeschreibung, Langbeschreibung und Marketplace Texte pruefen.
2. Screenshots fuer Setup, Dashboard, Scan und Report pruefen.
3. Keine DEV-Daten oder Secrets in Screenshots.

Erwartetes Ergebnis: Professionelle, aktuelle Listing Assets.

PASS-Kriterium: Assets vollstaendig und sauber.

FAIL-Kriterium: Screenshots fehlen, zeigen DEV-Daten oder veraltete UI.

Screenshot erforderlich: Ja

Log erforderlich: Nein

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

BC UI/XLF Legacy-Begriffe

IMPORTANT

Ziel: BC UI zeigt keine alten Produktbegriffe.

Voraussetzung: Aktuelle XLF und installierte App.

Schritte:

1. BC Pages visuell pruefen.
2. XLF/AL nach sichtbaren Free/Premium/Trial Begriffen durchsuchen.

Erwartetes Ergebnis: Nur Assessment, Validation Check, Monitoring Monthly und Monitoring Annual sichtbar.

PASS-Kriterium: Keine alten Kundentexte in BC UI.

FAIL-Kriterium: Free/Premium/Trial sichtbar.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Monitoring und Mail Reports

Monitoring Automation

IMPORTANT

Ziel: Monitoring-Mehrwert ist technisch und in der UI nachvollziehbar.

Voraussetzung: Monitoring Tenant vorhanden.

Schritte:

1. Geplante Scans/Scheduler pruefen.
2. Monitoring Access und wiederholte Scans pruefen.
3. Trends und Recent Scans pruefen.

Erwartetes Ergebnis: Monitoring funktioniert als wiederkehrender Produktmodus.

PASS-Kriterium: Automation und UI verhalten sich produktkonform.

FAIL-Kriterium: Keine Automation oder falsche Anzeige.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Mail Reports

NICE TO HAVE

Ziel: Automatisierte Mail Reports sind bewertet und nicht versehentlich aktiv unsicher.

Voraussetzung: Mail-Konfiguration und Testempfaenger vorhanden.

Schritte:

1. Empfaengerlisten, Opt-in und Versandkonfiguration pruefen.
2. Testmail senden, falls Feature aktiv ist.
3. Fehlerhandling und Datenschutz pruefen.

Erwartetes Ergebnis: Feature ist entweder sicher deaktiviert oder sauber testbar.

PASS-Kriterium: Kein ungewollter Versand, keine sensiblen Daten in Mails.

FAIL-Kriterium: Unkontrollierter Versand oder fehlendes Opt-in.

Screenshot erforderlich: Ja

Log erforderlich: Ja

Ergebnis:	PASS	FAIL	BLOCKED	NOT RUN
-----------	------	------	---------	---------

Notizen:

Pilot Go-Live Freigabe

Tenant registriert	OFFEN
Lizenz aktiv	OFFEN
Credits korrekt	OFFEN
Scan erfolgreich	OFFEN
Dashboard geprueft	OFFEN
Reports geprueft	OFFEN
Share Links geprueft	OFFEN
Open in BC geprueft	OFFEN

Pilot = GO / NO-GO

Notizen:

Customer Go-Live Freigabe

Assessment Kauf bestanden	OFFEN
Validation Kauf bestanden	OFFEN
Monitoring Monthly bestanden	OFFEN

Monitoring Annual bestanden	OFFEN
Webhook geprueft	OFFEN
Credits korrekt	OFFEN
Monitoring korrekt	OFFEN
Dashboard korrekt	OFFEN
Reports korrekt	OFFEN
Backup erfolgreich	OFFEN
Restore erfolgreich	OFFEN

Customer Go-Live = GO / NO-GO

Notizen:

AppSource Freigabe

AL Compile	OFFEN
CodeCop	OFFEN
UICop	OFFEN
AppSourceCop	OFFEN
Logo	OFFEN
Screenshots	OFFEN
Privacy URL	OFFEN
Terms URL	OFFEN
Help URL	OFFEN
Support URL	OFFEN
Marketplace Texte	OFFEN

AppSource = GO / NO-GO

Notizen:

Finale Freigabeseite

Gesamtentscheidung:

<i>Alle BLOCKER sind PASS</i>	OFFEN
<i>Alle CRITICAL Checks sind PASS oder mit akzeptiertem Risiko dokumentiert</i>	OFFEN
<i>Pilot Go-Live Freigabe ist GO</i>	OFFEN
<i>Customer Go-Live Freigabe ist GO</i>	OFFEN
<i>AppSource Freigabe ist GO</i>	OFFEN

Freigabe durch:

Datum:

Unterschrift / Entscheidung:

Offene Risiken:

Nächste Massnahmen:

Finale Freigabe

Handgefuehrter Pilot	GO	NO-GO	Datum / Name / Notizen
Erster zahlender Kunde	GO	NO-GO	Datum / Name / Notizen
AppSource / Marketplace	GO	NO-GO	Datum / Name / Notizen

Offene Risiken / Auflagen: